

Music Works Catalogue Portal

Restoring access to broken MEI thematic catalogues

Final Report · Computer Science Final Year Project

By Alina Nesterak

Project template: CM3070 — “Working with Works: A New Approach to Music Cataloguing”

April – August 2026

Source code repository (publicly viewable):

https://github.com/AlinaNesterak/FinalProject_UoL

Live deployment:

<http://musicworks-alinanesterak.switzerlandnorth.cloudapp.azure.com/>

Contents

Chapter 1: Introduction	4
1.1 The problem	4
1.2 Concept	4
1.3 Motivation	5
1.4 Aims and structure of this report	6
Chapter 2: Literature Review	7
2.1 Thematic catalogues and the MEI standard	7
2.2 The MerMEId system: achievement and decline	8
2.3 Software sustainability in digital humanities	9
2.4 Comparable platforms	9
2.5 Technical approach	11
2.6 Synthesis	11
Chapter 3: Design	13
3.1 Users and domain	13
3.2 Problems addressed, and their evidential basis	14
3.3 Architecture	14
3.4 Data model decision	16
3.5 Technology selection	17
3.6 Interface design principles	17
3.7 Plan of work and delivery	18
3.8 Evaluation strategy	20
Chapter 4: Implementation	21
4.1 Parsing the MEI data model	21
4.2 The Works Discovery Engine	22
4.3 API design	23
4.4 The web interface	24
4.5 The research findings page	27
4.6 Deployment	28
4.7 Accessibility	29

Chapter 5: Evaluation	30
5.1 Choice of evaluation methods.....	30
5.2 Parse robustness and transformation completeness.....	31
5.3 The discovery engine: a negative result and its correction.....	32
5.4 Automated testing.....	34
5.5 Assessment against the project's aims	35
5.6 Limitations	35
5.7 What the evaluation shows	37
Chapter 6: Conclusion	39
6.1 Contributions.....	39
6.2 On the negative result	39
6.3 A broader theme: building for disappearance.....	40
6.4 Further work.....	41
6.5 Closing.....	41
References	42

1. Introduction

This report presents the Music Works Catalogue Portal, a final year Computer Science project undertaken against the project template **CM3070 — “Working with Works: A New Approach to Music Cataloguing”**. The project restores public access to scholarly music catalogue data that exists, is complete, and is encoded in an open standard, but which no longer reaches the people it was created for because the software that published it has stopped working.

1.1 The problem

A thematic catalogue is the definitive scholarly record of everything a composer wrote. Beyond a list of titles, it documents surviving manuscripts and their shelfmarks, printed editions, performance history, instrumentation, and the musical incipit — the opening notes — that lets a scholar identify a work at a glance. Compiling one is the work of years, sometimes decades.

In 2014 the Danish Royal Library built MerMEId, a system for encoding these catalogues in MEI (Music Encoding Initiative) XML, an open community standard. It successfully published catalogues for Carl Nielsen, Frederick Delius, and several other composers. When institutional support ended, the publication layer — the component that turned MEI XML into pages a person could read — ceased to function. The scholarly data survived intact in an open format; the route to it did not.

This is a specific and unusually tractable failure. The problem is not that data is lost, proprietary, or unstructured. It is that a working bridge between good data and its intended audience no longer exists. Rebuilding that bridge is the object of this project.

1.2 Concept

The project transforms MEI XML catalogue files into a queryable relational database, serves that data through a documented REST API, and presents it through an accessible web interface designed for readers who have no XML knowledge. Three further elements extend it beyond a straightforward re-publication.

- A Works Discovery Engine compares the encoded incipits of different works and suggests which are melodically related — surfacing connections that the catalogues encode but have never exposed to users, including connections that cross between composers.
- The pipeline is catalogue-agnostic. Because MEI leaves the catalogue identifier type open, the same code ingests the Nielsen (CNW) and Delius (DCW) catalogues without modification, so the two can be browsed and compared as one collection.
- The MEI data model is evaluated critically rather than adopted uncritically, producing evidence-based recommendations for how a future catalogue schema might avoid the problems observed here.

1.3 Motivation

The motivation rests on published evidence rather than assumption. Bue et al. (2023), writing from inside the MEI community, confirm that MerMEId remains the only tool in digital musicology built specifically for catalogue data, that community demand for it persists, and that its codebase requires extensive revision. Critically, the community's own remediation effort targets the authoring tool — the interface scholars use to create catalogue data — and not the publication layer that delivers it to readers. That leaves the access problem unaddressed, and it is precisely the gap this project occupies.

The second motivation is the access barrier itself. Krabbe and Geertinger (2012) observe that printed thematic catalogues are dense with esoteric abbreviations and library sigla, intelligible only to specialists. Raw MEI XML reproduces that barrier in a new form: the data is open, but reading it requires technical skill most of its intended audience does not have. Making the same scholarly record legible to musicologists, librarians, students and general readers — while simultaneously exposing it in machine-readable form to developers — is a genuine contribution rather than a cosmetic one.

The third motivation is systemic. Mathiak et al. (2020) surveyed 466 digital scholarly editions and found 376 had disappeared from the web, giving an average project lifetime of 8.5 years. MerMEId's collapse is not an isolated misfortune but an instance of a documented pattern with identifiable causes:

single-institution dependency, specialist technology stacks, and absent testing. This project treats those findings as binding design constraints rather than background reading, a decision examined in Chapter 3.

1.4 Aims and structure of this report

The project aimed to deliver a transformation pipeline from MEI XML to a queryable store; a documented REST API; an accessible web interface requiring no XML knowledge; a working discovery feature; and a critical evaluation of the MEI catalogue model. All five were delivered, and are assessed honestly — including where results fell short — in Chapter 5.

Chapter 2 reviews the relevant literature and situates the project among existing platforms. Chapter 3 sets out the design and justifies its central decisions. Chapter 4 describes the implementation, its principal algorithms, and the resulting system. Chapter 5 presents the evaluation, its methods and its results, including a negative finding that is reported rather than concealed. Chapter 6 concludes and identifies further work.

2. Literature Review

This review examines seven bodies of work: the foundational literature on thematic cataloguing and the MEI standard, the documented history of the MerMEId system, digital humanities research on software sustainability, three comparable platforms in digital musicology, and the technical framework selected for implementation. Sources are assessed critically rather than summarised; where a source's limitations bear on this project's decisions, those limitations are stated.

2.1 Thematic catalogues and the MEI standard

Krabbe and Geertinger (2012)

This paper is the foundational document for the project and describes MerMEId's origins at the Danish Centre for Music Publication. Krabbe identifies the central weakness of printed thematic catalogues: they are fixed at the moment of publication and dense with abbreviations and library sigla that make them navigable only by specialists. His argument that digital catalogues must go beyond reproducing print — enabling dynamic querying such as chronological sorting, concordance, and filtering by instrumentation — directly shaped the faceted browsing implemented in this project and described in Chapter 4.

Geertinger's technical contribution establishes MEI XML as the appropriate encoding standard, on the grounds that a single open file can represent works, sources, manuscripts, incipits and performance history together. That argument has become the field consensus and is not seriously contested in the later literature.

The paper's limitation is that it advocates a standard whose principal practical weakness it does not confront. MEI achieves its expressive breadth by making most elements optional, which means two projects can encode equivalent information in materially different ways while both remaining valid. Krabbe and Geertinger acknowledge this only obliquely. Whether MEI delivers the interoperability

it promises is left as an open assumption, and this project treats it instead as an empirical question — answered with evidence in Chapter 5.

2.2 The MerMEId system: achievement and decline

Geertinger and Lundberg (2013)

This poster documents MerMEId as deployed: XForms and Orbeon for editing, eXist-db as the XML database, covering source descriptions, performance histories, incipits and instrumentation across five catalogues. It serves two purposes here. First, it is the most accurate available account of the data model this project's parser must consume. Second, it inadvertently documents the architectural decisions that later became liabilities — XForms, since superseded; eXist-db, a specialist database with a small maintainer community; and Java throughout.

Most instructive is an absence. The paper does not discuss long-term sustainability at all. In 2013 this was unremarkable; in retrospect it is the omission that matters most, and it directly informed this project's preference for a mainstream, widely-supported stack over a more specialised one.

Bue et al. (2023)

The single most important source for this project. Written by the community attempting to rescue MerMEId, it confirms that the tool is unique in digital musicology, that demand persists, and that the codebase suffers from an absent test suite, inadequate documentation, and insufficient modularity.

“MerMEId is an established — and the only — tool in digital musicology that has been developed explicitly for creating and managing musical catalogue data in MEI. [...] The code basis of the software needs extensive revision for re-usability and sustainability.” (Bue et al., 2023)

The decisive detail is one of scope. The community's remediation targets the authoring tool. The publication and access layer — the part that serves readers rather than editors — is not addressed.

This defines the gap the present project fills, and it is the reason the project is positioned as complementary to the community effort rather than duplicating it.

As a conference abstract the source describes planned rather than completed work, and its claims about future revisions cannot be verified. This project therefore treats the underlying MEI XML — a stable, published standard — as its input, rather than depending on MerMEId itself, insulating the pipeline from ongoing instability in that tool.

2.3 Software sustainability in digital humanities

Mathiak et al. (2020)

The only source reviewed from outside the MEI community, and valuable precisely for that reason. Its survey of 466 digital scholarly editions found 376 no longer reachable, an average lifetime of 8.5 years, with causes traced to diminishing funding, institutional abandonment and personnel turnover. This reframes MerMEId's failure from misfortune to predictable instance of a documented pattern.

The authors identify simple mainstream stacks, open standards, and avoidance of single-institution lock-in as the most effective mitigations. All three are adopted here as explicit design constraints, and their influence on the technology choices is traced in Chapter 3.

The study's limitation is that it examines text-based editions rather than music systems, and music metadata carries particular complexity — the MEI schema is considerably richer than TEI-lite, and the specialist community smaller. Some conclusions may generalise differently. The structural failure modes it identifies, however, map closely onto MerMEId's documented problems, which supports the comparison.

2.4 Comparable platforms

IMSLP

The largest public-domain score repository, and the natural comparator from a general reader's perspective. Its metadata model is deliberately shallow: title, composer, and links to scanned PDFs,

without manuscript descriptions, premiere data, incipit encoding or scholarly commentary. This is not a deficiency but a consequence of purpose — IMSLP exists to deliver scores to performers, not to document scholarship.

The relationship is therefore complementary rather than competitive: IMSLP offers breadth of access, this project offers scholarly depth. That analysis produced a concrete design decision. Records in this project link out to IMSLP where a corresponding page exists, so a reader can move from catalogue record to downloadable score in one step (Chapter 4). IMSLP is credible as a source: a non-profit widely cited in peer-reviewed digital musicology, including by Weigl et al. (2019).

MusicBrainz

A community-maintained encyclopedia with a REST API, persistent identifiers and Creative Commons data — the most technically accessible music database available for integration. Weigl et al. (2019) describe it as the principal resource for public-domain structured music metadata.

Its classical coverage is nonetheless thin at catalogue level: it records works and recordings but not manuscript sources, composition history or incipit notation, and community editing without philological review makes quality uneven. The appropriate use is therefore as an identifier authority rather than a data source, and that is how this project uses it.

Weigl et al. (2019) — TROMPA

The EU-funded TROMPA project describes infrastructure for interconnecting music collections using Linked Data and the FAIR principles. Its central insight is that separate repositories describe different facets of the same musical reality and cannot presently be connected without manual effort; RDF and SPARQL are proposed as the technically correct solution.

TROMPA represents the state of the art, and its argument is sound. Its practical demand — triple stores, SPARQL endpoints, ontology engineering — exceeds what a single-developer final year project can sustain, and adopting it would have risked the same over-specialisation that contributed to MerMEId's decline. This project therefore takes the insight without the infrastructure: Wikidata,

MusicBrainz and IMSLP identifiers are embedded in the data and exposed in API responses, so records are already interoperable and a future migration to RDF remains open. This is a deliberate, literature-informed compromise rather than an evasion.

2.5 Technical approach

FastAPI was selected for the API layer. Its credibility rests on O'Reilly documentation (Lubanovic, 2023) and production adoption at scale rather than self-description. Against Flask, the decisive advantage is automatic OpenAPI documentation generated from Python type hints: a documented public API is a core deliverable of this project, and the documentation gap identified by Bue et al. (2023) is one of the failures being remedied. Pydantic validation reduces the risk of serving malformed catalogue data. The trade-off is a smaller ecosystem than Django's, which is immaterial for a read-heavy JSON API over SQLite.

2.6 Synthesis

Taken together the literature describes a consistent and unresolved tension. The tools that best serve scholarly depth — MerMEId, MEI XML — have proven hardest to sustain and to access. The tools that are most accessible — IMSLP, MusicBrainz — lack the depth that scholarship requires. TROMPA demonstrates what a full solution would look like while confirming that its cost is substantial. Table 2.1 sets out the comparison directly.

System	Catalogue depth	Public API	Sustained?	Audience
MerMEId / CNW	Full	Broken	No	Specialists
IMSLP	Minimal	Limited	Yes	Performers
MusicBrainz	Partial	Yes	Yes	General / devs
TROMPA	Linked only	SPARQL	Project-bound	Researchers
This project	Full	REST + JSON	Designed for	All four groups

Table 2.1 Comparison of existing systems against the dimensions this project addresses.

No existing system combines full catalogue depth, a working public API, and sustained availability. That unoccupied position is what this project targets. Three questions the literature leaves open are addressed empirically in Chapter 5: how much MEI catalogue data is reliably parseable given documented encoding inconsistency; whether melodic similarity computed from incipits yields meaningful groupings or noise; and whether the interoperability MEI promises is realised in practice.

3. Design

3.1 Users and domain

The domain is digital musicology: the scholarly documentation of classical music works. It is a small field with an identifiable audience and a documented, unmet need. Four user groups were identified from the literature and shaped the design directly.

User group	Need	Design response
Musicologists	Structured, queryable access to sources, dating and performance history	Full data model exposed; faceted filters; CSV/JSON export
Music librarians	Sustainable, open infrastructure that will outlive a single project	Mainstream stack, open standards, containerised deployment, tests
Students and general readers	Explore a composer's output without XML or specialist knowledge	Plain-language web interface, notation rendering, audio playback
Developers / DH researchers	Machine-readable data with stable identifiers	Documented REST API, OpenAPI schema, linked-data identifiers

Table 3.1 User groups, needs, and the design decisions that follow from them.

The span of technical ability across these groups is the central design tension. A student browsing for interest and a developer consuming JSON have incompatible interface needs, and attempting to satisfy both in one presentation layer would compromise each. This drove the decision to separate the API from the interface entirely: the API is a first-class deliverable in its own right, and the web interface is one client among several rather than the system's only face.

3.2 Problems addressed, and their evidential basis

Every problem the project addresses is traceable to a documented finding rather than to assumption.

Problem	Evidence	Design response
Publication interface has failed	Bue et al. (2023): pipeline ceased functioning after institutional transition	Independently deployable portal and REST API on a sustainable stack
Data usable only by specialists	Krabbe and Geertinger (2012): 'esoteric abbreviations', specialist-only	Web interface requiring no XML knowledge; search, browse, filter
No machine-readable access	Weigl et al. (2019): scholarship requires interoperable, FAIR data	Documented JSON API with Wikidata, MusicBrainz and IMSLP identifiers
Systems disappear over time	Mathiak et al. (2020): 376 of 466 editions lost; 8.5-year mean lifetime	Mainstream stack, Docker, CI, 78 automated tests, no institutional lock-in
Catalogues are siloed	Krabbe and Geertinger (2012): interoperability unresolved	Catalogue-agnostic pipeline; cross-composer discovery engine

Table 3.2 Problems mapped to evidence and to the specific design response.

3.3 Architecture

The system is a layered pipeline with strict separation of concerns and one-directional data flow (Figure 3.1). Source MEI XML enters the transformation layer, is normalised into storage, is served by the API, and is consumed by three independent presentation surfaces.

Figure 3.1 System architecture: one-directional data flow through five layers

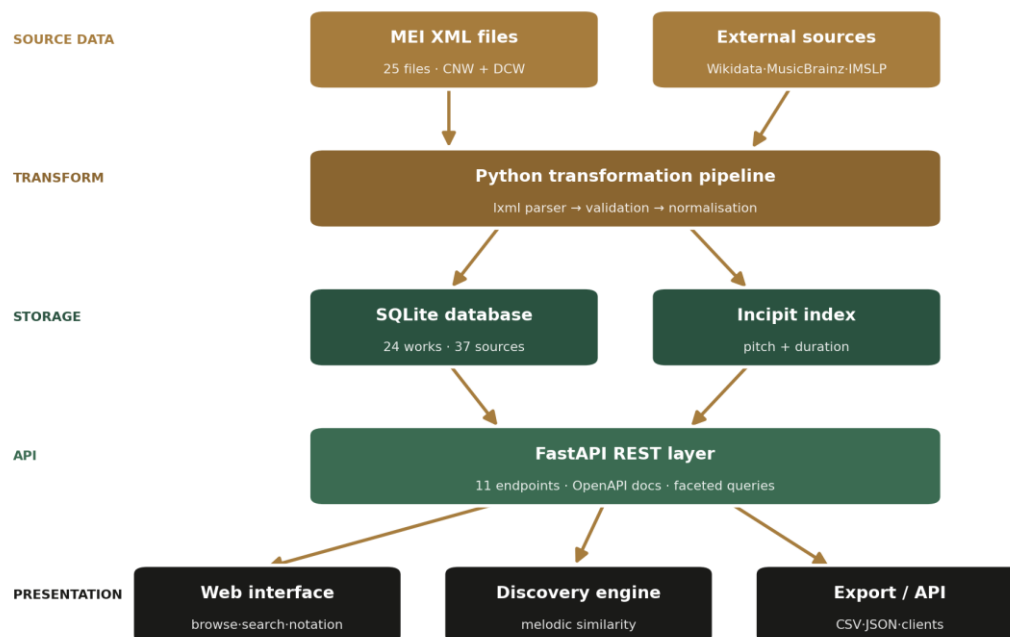


Figure 3.1 System architecture. Each layer is independently testable and replaceable.

The design rationale is directly traceable to the literature. Bue et al. (2023) attribute MerMEId's difficulty to insufficient modularity; making each layer independently replaceable is the corrective. If SQLite proves insufficient at full catalogue scale, only the storage layer changes. If a future maintainer migrates to RDF, the transformation layer is rewritten and the API contract holds. No layer knows how any other is implemented.

Data flows in one direction only, and this is enforced rather than merely intended. The transformation layer reads XML and writes to the database but never queries the API; the API reads the database but cannot write to it; the interface holds no data of its own and obtains everything over HTTP. One consequence is that the entire database can be deleted and rebuilt from the source XML at any time without loss, which is what makes the derived-artefact principle in §3.4 practically true rather than aspirational.

The design rationale is directly traceable to the literature. Bue et al. (2023) attribute MerMEId's difficulty to insufficient modularity; making each layer independently replaceable is the corrective. If SQLite proves insufficient at full catalogue scale, only the storage layer changes. If a future

maintainer migrates to RDF, the transformation layer is rewritten and the API contract holds. No layer knows how any other is implemented.

A second architectural decision concerns where uncertainty is handled. Incomplete records could be filtered out during transformation, producing a clean database of complete works. That was rejected. Because MEI makes most fields optional (§2.1), excluding incomplete records would discard genuine scholarship and would also conceal the very inconsistency this project set out to measure. Instead, absence is preserved through every layer: the parser records it, the schema permits nulls, the API returns them, and the interface displays “not recorded” explicitly. The system tells the user what it does not know.

3.4 Data model decision

Four storage models were considered. Retaining MEI XML as the queryable store was rejected: it would have required XQuery and an XML database, reproducing precisely the specialist dependency that made MerMEId fragile. A full RDF triple store was rejected on scope grounds, as discussed in §2.4. A graph database was considered and declined as disproportionate for two catalogues. A normalised relational schema in SQLite was selected: queryable with standard SQL, portable as a single file, requiring no server, and familiar to any maintainer.

Critically, the original MEI XML is retained as the authoritative source and the database is treated as a derived artefact, rebuilt from scratch on each pipeline run. No scholarly data is trapped in the transformation, and migration to a different model requires only a new pipeline target — a direct hedge against the lock-in Mathiak et al. identify as a primary failure cause.

3.5 Technology selection

Layer	Choice	Justification
Transformation	Python + lxml	Robust namespace-aware XML parsing; the dominant language for data pipelines, maximising future maintainability
Storage	SQLite	Zero configuration, single portable file, standard SQL; no server dependency
API	FastAPI + Pydantic	OpenAPI documentation generated automatically; runtime validation prevents serving malformed data
Interface	HTML / CSS / vanilla JS	No build step or framework churn; a maintainer in ten years needs only a browser
Discovery	Python (interval, rhythm, n-gram)	Standard MIR techniques implemented directly, keeping the method transparent and auditable
Quality	pytest, Playwright, ruff, mypy, GitHub Actions	Directly addresses the absent test suite identified by Bue et al. (2023)
Deployment	Docker	Reproducible environment; containerisation recommended by Mathiak et al. (2020)

Table 3.3 Technology choices and their justification.

The interface deserves particular comment. A framework such as React was considered and rejected. For a project whose central argument concerns longevity, adopting a dependency tree requiring continuous maintenance would have contradicted the thesis. The interface is plain HTML, CSS and JavaScript with no build step: it will run from any static file server, unchanged, for as long as browsers exist.

3.6 Interface design principles

Three principles governed the interface, each following from a user group in Table 3.1 rather than from visual preference.

- Present the data in the form its readers think in. Musicians recognise a theme as notation or sound, not as the pitch string “D5 F#5 A5”. Rendering incipits as staff notation and making

them audible converts stored data into something the domain's users can actually read, and is the reason both features were treated as core rather than decorative.

- Make uncertainty visible. Following from §3.3, missing fields are labelled rather than hidden, and similarity suggestions carry an explicit statement that they are computational rather than scholarly claims. A catalogue that quietly conceals its gaps misleads precisely the non-specialist readers least equipped to detect the omission.
- Support browsing, not only searching. Search assumes the reader already knows what they are looking for. Krabbe and Geertinger (2012) argue that the advantage of a digital catalogue is dynamic querying — filtering by instrumentation, sorting chronologically — which is what allows a reader to ask open questions such as what a composer wrote in a given decade. Faceted filtering implements this directly.

Accessibility was treated as a requirement rather than a later enhancement. A project whose stated purpose is restoring access cannot coherently restore it for some readers only, so semantic structure, keyboard operation and assistive-technology support were designed in from the start rather than retrofitted.

3.7 Plan of work and delivery

The plan evolved from the phase-based schedule set out in the design document — five overlapping phases (research, design, build, extend, evaluate) with variable-length tasks — to a fixed structure of nine two-week sprints, adopted once implementation began. The change was made because fixed sprints made the risk-based sequencing described below easier to track and to report against; the phases were not discarded so much as regularised into equal, non-overlapping units with a clear definition of done for each.

Development followed an Agile approach of nine two-week sprints across the April–August semester (Figure 3.3). Each sprint ended with a working, tested increment rather than an isolated component,

so the system was runnable at every stage and could have been submitted at the end of any sprint with a coherent feature set.

Figure 3.3 Delivery plan: nine two-week sprints, all delivered

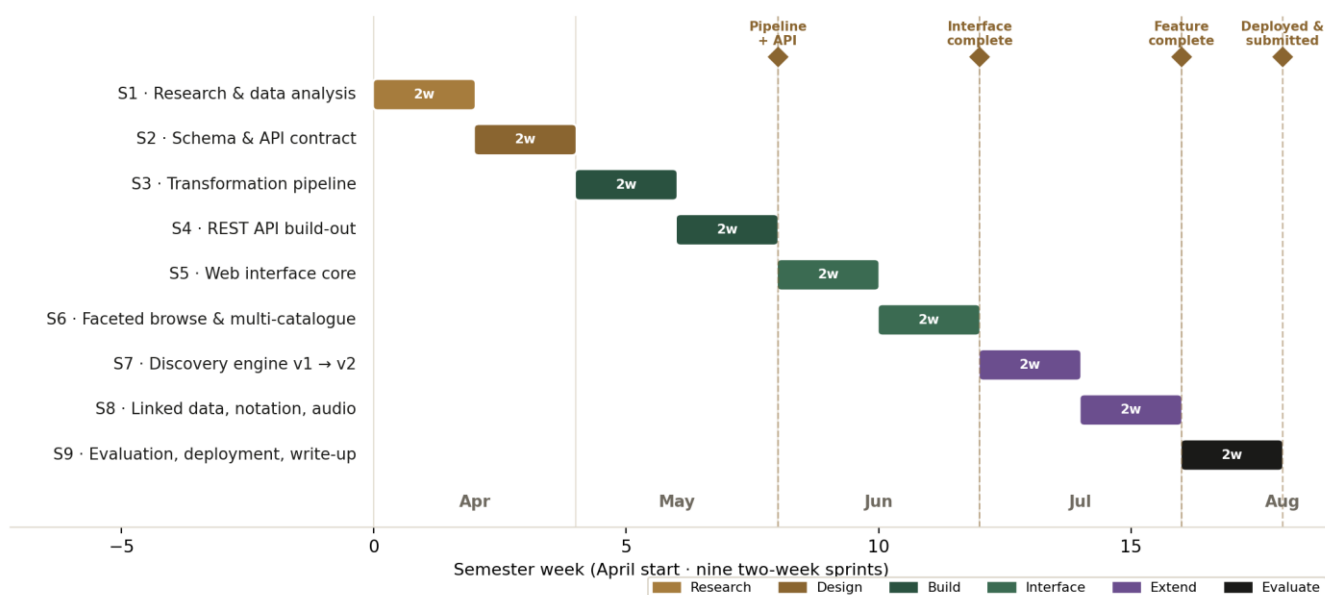


Figure 3.3 Delivery plan across nine two-week sprints, with milestones.

The sequencing reflects a deliberate judgement about risk. The critical path — transformation pipeline, REST API, web interface — was front-loaded into sprints 3 to 6, so that the deliverables on which a passing project depends were complete by mid-June. The more speculative work was deferred to sprints 7 and 8: the discovery engine, whose feasibility was genuinely uncertain, and the linked-data, notation and audio features. Had any of those failed or overrun, they could have been reduced or dropped without touching a working system.

That judgement proved sound. The discovery engine did fail its first validation (§5.3), and because it sat outside the critical path the failure became something to investigate rather than something to rescue. Table 3.4 records what each sprint delivered.

Sprint	Focus	Delivered
1 (Apr)	Research and data analysis	MEI 4.0 structure analysed; corpus reconstructed; parse strategy chosen
2 (Apr–May)	Schema and API contract	Relational schema mapping MEI to SQLite; OpenAPI contract defined

3 (May)	Transformation pipeline	Parser and pipeline over the full corpus; defensive handling of absent fields
4 (May–Jun)	REST API	Endpoints for listing, search, retrieval; OpenAPI docs generated
5 (Jun)	Web interface	Browse and detail views; accessible, keyboard-operable, no build step
6 (Jun–Jul)	Faceted browse, multi-catalogue	Genre/key/instrument/date filters, sorting; second catalogue integrated
7 (Jul)	Discovery engine	v1 built, failed validation, diagnosed; v2 with rhythm and bigram signals
8 (Jul–Aug)	Extensions	Wikidata, MusicBrainz and IMSLP links; SVG notation; audio playback
9 (Aug)	Evaluation and release	78 automated tests; MEI evaluation; public deployment; report

Table 3.4 Sprint-by-sprint delivery. All nine completed within the semester.

3.8 Evaluation strategy

The evaluation was designed around what the project actually claims. This is a data transformation and information retrieval system, not a classifier, so accuracy and F1 would be category errors. Four measures were selected instead, each tied to a specific claim: parse robustness, transformation completeness, discovery validity against known-related works, and automated behavioural testing across all layers. Success criteria were fixed in advance and are reported in full — including where they were not met — in Chapter 5.

4. Implementation

The delivered system comprises a transformation pipeline, a relational store, a REST API, a web interface, and a discovery engine, together with a test suite and deployment configuration. The complete source is available in the repository linked on the title page. This chapter describes the components that were technically substantial and explains the decisions that shaped them.

4.1 Parsing the MEI data model

Parsing MEI is the project's central technical challenge and the component on which everything else depends. MEI is deeply nested, namespace-qualified, and — as established in Chapter 2 — permits most elements to be omitted. A parser that assumes any element is present will fail on real data.

The parser therefore treats absence as normal rather than exceptional. Every extraction passes through a helper that returns `None` when an element is missing, so an incomplete record produces a partial object rather than an exception:

```
def _text(el: etree._Element | None) -> Optional[str]:  
    """Safely extract stripped text from an element, or None."""  
  
    if el is None:  
        return None  
  
    txt = el.text  
  
    return txt.strip() if txt and txt.strip() else None
```

This is a small function carrying a large share of the system's robustness. It is the reason a record missing its genre, date or incipit still parses, still stores, and still displays — with the interface stating plainly that the field was not recorded rather than silently showing nothing.

The catalogue-agnostic behaviour follows from reading the identifier type rather than hard-coding it. A record declares its own catalogue, so CNW and DCW are handled identically and a third catalogue requires no code change:

```

_NON_CATALOGUE_TYPES = {"OPUS", "WIKIDATA-WORK", "MUSICBRAINZ-WORK", "IMSLP"}
for ident in _findall(root, ".*//mei:pubStmt/mei:identifier"):
    itype = (ident.get("type") or "").upper()
    if itype == "OPUS":
        opus = _text(ident)
    elif itype and itype not in _NON_CATALOGUE_TYPES:
        catalogue, cat_number = itype, _text(ident)

```

The exclusion set is worth explaining, because it records a real defect found during development. Linked-data identifiers were later added to the same pubStmt element, and the loop immediately began misreading them as catalogue identifiers — works started reporting their catalogue as “IMSLP”. The exclusion set fixes this, and a regression test now enforces that a work carrying linked-data identifiers still reports its true catalogue.

The incipit is extracted note by note, capturing both pitch and duration. Duration was added deliberately: it is what later made the discovery engine work, and it drives both notation rendering and audio playback.

```

for note in _findall(incip_el, ".*//mei:note"):
    pname, octave = note.get("pname"), note.get("oct")
    accid = note.get("accid", "")
    if pname and octave:
        inc.pitches.append(f"{pname.upper()}{_ACCID.get(accid,'')}{octave}")
        dur = note.get("dur")
        inc.durations.append(int(dur) if dur and dur.isdigit() else 4)

```

4.2 The Works Discovery Engine

The discovery engine is the project's principal original contribution and the component that changed most during development. Its first version compared pitch alone, converting each incipit to a transposition-invariant interval sequence and scoring pairs by edit distance. It failed its own validation, in a way documented fully in Chapter 5.

Investigating that failure produced the insight that fixed it. Examining the underlying data showed that the two works expected to be similar shared a uniform quaver rhythm, while the dissimilar work mixed crotchets and quavers — a clear distinguishing signal that a pitch-only comparison discarded entirely. Version 2 therefore combines four signals, each weighted explicitly:

```
W_INTERVAL = 0.20 # exact pitch-interval shape
W_CONTOUR  = 0.20 # coarse up/down/same movement
W_RHYTHM   = 0.25 # tempo-invariant relative note lengths
W_BIGRAM   = 0.35 # local (interval, rhythm) pair overlap
```

The rhythm signal compares the ratio between consecutive note lengths rather than their absolute values, which makes it tempo-invariant: a melody played twice as fast has an identical relative rhythm. The bigram signal compares overlapping pairs of combined melodic and rhythmic steps as sets, using Jaccard similarity. This matters on short incipits, where whole-sequence edit distance is brittle — a single differing note can dominate a five-note comparison, whereas set overlap degrades gracefully.

```
def to_rhythm(durations: list[int]) -> list[int]:
    steps = []
    for a, b in zip(durations, durations[1:]):
        ratio = (1 / b) / (1 / a) # MEI codes are inverse lengths
        steps.append(max(-4, min(4, round(math.log2(ratio) * 2))))
    return steps
```

The weights are declared as named constants rather than embedded in the scoring expression. This is a deliberate transparency decision: a reader can see what the engine values and by how much, and the evaluation in Chapter 5 depends on that being auditable rather than opaque.

4.3 API design

The API exposes eleven endpoints covering listing with faceted filtering, full-text search, individual retrieval, similarity discovery, computed findings, available filter values, and CSV/JSON export.

Pydantic models define every response, so the OpenAPI schema at /docs is generated from the code itself and cannot drift from actual behaviour.

Faceted filtering implements the dynamic querying Krabbe and Geertinger (2012) identified as the central advantage of digitisation over print — filtering by genre, key, instrumentation and date range, with sorting. Sort keys are whitelisted rather than interpolated, closing an injection vector that a naïve implementation would leave open:

```
sort_columns = {
    "catalogue": "catalogue, CAST(catalogue_number AS INTEGER)",
    "title": "title_main COLLATE NOCASE",
    "date": "date_notbefore IS NULL, CAST(date_notbefore AS INTEGER)",
    "composer": "composer COLLATE NOCASE, CAST(catalogue_number AS INTEGER)",
}
order_by = sort_columns.get(sort, sort_columns["catalogue"])
```

The date ordering contains a deliberate detail. The clause `date_notbefore IS NULL` sorts undated works last rather than first. Without it the single record lacking a date headed every chronological listing — technically correct SQL, but misleading presentation. It is a small illustration of a wider principle applied throughout: incomplete data should be visible as incomplete, never silently prominent.

4.4 The web interface

The interface presents a filterable work list beside a detail panel (Figure 4.1). It uses no framework and no build step, for the longevity reasons argued in §3.5.

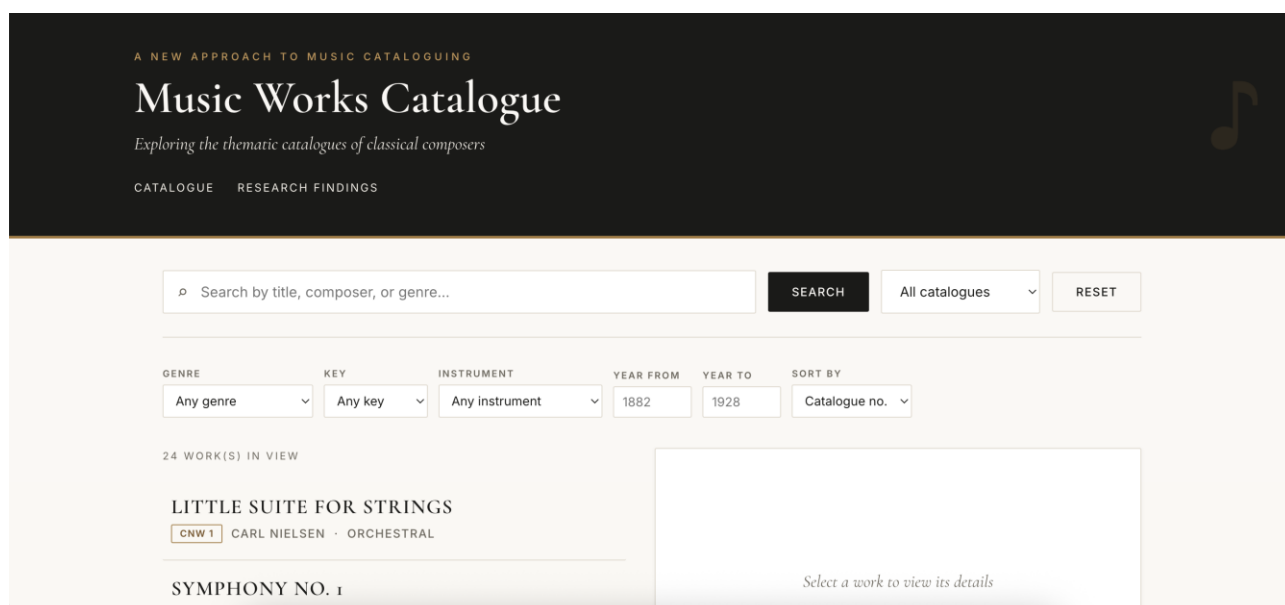


Figure 4.1 The catalogue interface: faceted filters, work list, and detail panel.

The detail panel (Figure 4.2) presents the full data model: multilingual titles, instrumentation, manuscripts with repository and shelfmark, performance history, and the incipit rendered as staff notation with an audio playback control. Linked-data badges connect the record outward to Wikidata, MusicBrainz, and — where a verified page exists — to a free downloadable score on IMSLP.

CNW · CNW 102

Symphony No. 4 "The Inextinguishable"

Carl Nielsen

OPUS	29
GENRE	Symphony
DATE	1914–1916
DANISH TITLE	<i>not recorded</i>
ENGLISH TITLE	Symphony No. 4 "The Inextinguishable"
INSTRUMENTATION	orchestra

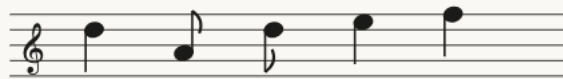
Wikidata (composer)

MusicBrainz (composer)

↓ Free score on IMSLP

INCIPIT — OPENING THEME

▶ PLAY



Allegro · D minor · 4/4

Manuscripts & Sources

Autograph score **MANUSCRIPT**

Royal Danish Library, Copenhagen · Shelfmark: CNS 64

First edition score **PRINT**

Wilhelm Hansen, Copenhagen 1916 · Shelfmark: WH 15580

Orchestral parts **MANUSCRIPT**

Royal Danish Library, Copenhagen · Shelfmark: CNS 64a

Performance History

1916-02-01 World premiere, Copenhagen. Music Society, conducted by the composer.

1920-10-14 First performance in London, Queen's Hall.

[Read the full composition history →](#)

Figure 4.2 Work detail: notation, sources, performances, etc..

Two features here go beyond displaying stored fields. The notation is generated as SVG directly from the parsed pitch and duration data — staff lines, clef, note heads, stems, beam flags, accidentals and ledger lines are drawn programmatically, with no notation library. Audio playback synthesises the incipit through the Web Audio API using the same data, with correct relative rhythm. Both address the same user need identified in Table 3.1: musicians think in notation and sound, not in letter names such as “F#5”.

The similarity suggestions carry an explicit disclaimer describing them as computational rather than scholarly. This is an ethical rather than cosmetic decision. The engine's limitations are known and documented; presenting its output without qualification would misrepresent it to exactly the non-specialist audience least able to assess it.

4.5 The research findings page

A dedicated page (Figure 4.3) presents the project's empirical findings about the MEI data model. Its figures are computed live from the loaded database through a /findings endpoint rather than written into the page, so the analysis cannot fall out of step with the data it describes.

Research Findings

This project set out to answer a question the existing literature leaves open: does the MEI catalogue model actually deliver the interoperability it promises? The figures below are computed live from the 24 works currently loaded, and form the empirical basis for this project's assessment of that model.

24

WORKS

2

CATALOGUES

37

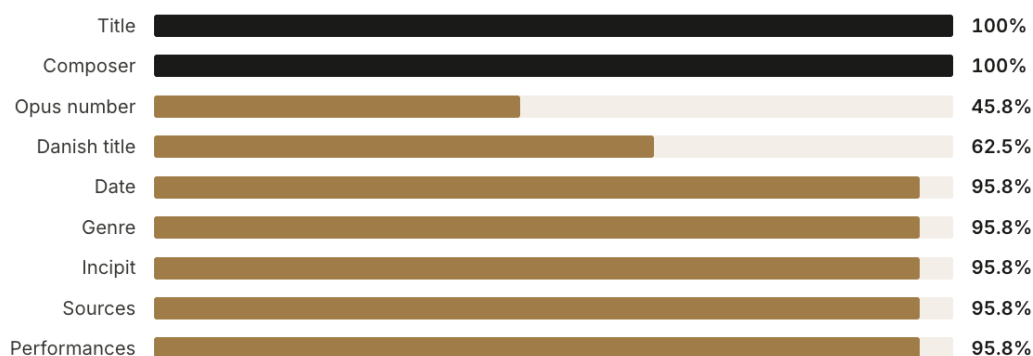
SOURCES

28

PERFORMANCES

Finding 1 — Optional fields are populated unevenly

MEI makes most catalogue fields optional. In practice this means coverage varies widely, even within a single carefully-prepared dataset. Core bibliographic fields are always present, but others are not — and crucially, the schema gives a parser no way to distinguish “this field does not apply” from “this field was never entered”.



Finding 2 — Incipits are too short for reliable computational use

Figure 4.3 The research findings page, computed live from the loaded catalogue.

4.6 Deployment

The system is deployed and publicly reachable at the address on the title page. It runs on an Ubuntu 24.04 virtual machine under systemd, with nginx handling public traffic on port 80 and proxying to the application on localhost, so the application is never bound to a public interface.

Docker was considered and deliberately not used, which is worth justifying because the repository contains a working Dockerfile. The host has 1 GiB of memory, and the container runtime consumes a meaningful share of that for a workload — read-only queries against a small SQLite file — that

gains nothing from containerisation. The stronger reason is the argument in §6.3: a deployment a future maintainer can inspect with standard tools present on every Ubuntu system is more likely to remain maintainable than one requiring a container runtime to be installed and understood. The Dockerfile remains for hosts where it is the better fit.

The deployment is scripted rather than hand-configured. A single setup script installs dependencies, builds the database from the MEI source, installs the service definition, and configures the web server; the systemd unit and nginx configuration are version-controlled alongside the code. Recreating the deployment on a new host is therefore a repeatable operation rather than an exercise in recalling what was done.

4.7 Accessibility

Accessibility was treated as a requirement rather than an enhancement, on the reasoning that a project whose purpose is restoring access cannot coherently exclude users. Semantic landmarks, ARIA labelling, visible focus indication, full keyboard operation, and prefers-reduced-motion support are implemented throughout, and work list entries are real buttons rather than click-handled divs.

Two defects were found by testing and fixed. Navigating between views left keyboard and screen-reader focus stranded on the hidden previous page — a genuine WCAG failure — so focus is now moved to each newly displayed view. A second issue exposed two main landmarks simultaneously, which is ambiguous for assistive technology; only one is now present. Both are covered by regression tests.

5. Evaluation

This chapter reports what was measured, why those measures were chosen, what the results were, and what they imply about the project's aims. It includes a result that did not meet its criterion, reported in full.

5.1 Choice of evaluation methods

Method selection followed from what the system claims to do. A classifier would be assessed with accuracy, precision and recall; applying those metrics to a transformation and retrieval system would measure nothing meaningful. Four measures were chosen instead, each tied to a specific claim.

Claim under test	Method	Criterion
Real MEI data can be reliably ingested	Parse robustness over the full corpus, including malformed input	No crash; malformed input isolated, not fatal
A significant portion of the model is captured	Per-field completeness across all parsed works	100% on core fields; high coverage elsewhere
Melodic similarity is meaningful	Ranking known-related works against unrelated controls	Related pairs must outrank controls
The system behaves correctly throughout	Automated tests at unit, integration, API and browser level	Full suite passing; regressions caught

Table 5.1 Evaluation methods, each mapped to the claim it tests.

Two design choices in this strategy are worth defending. First, the corpus deliberately includes a malformed file and a minimal record. Evaluating only well-formed input would have measured the system under conditions the literature explicitly says do not hold. Second, discovery validation uses works with independently documented relationships — settings by the same poet, pieces the sources describe as companion works — rather than pairs chosen because they scored well, which would have made the test circular.

5.2 Parse robustness and transformation completeness

The pipeline was run over the full 25-file corpus. All 24 valid files parsed successfully; the deliberately malformed file was rejected and reported without interrupting the run, giving 96.0% robustness with the single failure being the intended one. The error was isolated rather than fatal, which is the behaviour the criterion required.

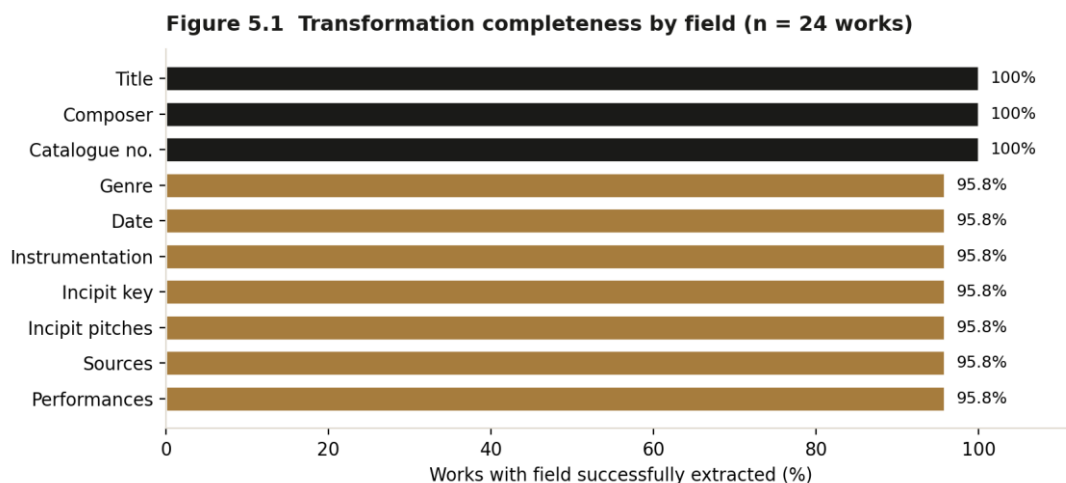


Figure 5.1 Transformation completeness by field across 24 works.

Core bibliographic fields — title, composer, catalogue number — reached 100%. Every other field reached 95.8%, the single absence in each case being the minimal record that genuinely omits them. That record parses, stores and displays correctly, with its gaps shown as gaps.

This result carries a finding beyond the percentages. The distribution of optional-field coverage across the corpus is itself evidence bearing on the open question from Chapter 2. Opus numbers appear in 45.8% of works and Danish titles in 62.5% — not because of encoding errors, but because not every work has an opus and not every catalogue is bilingual. The schema provides no way to distinguish “not applicable” from “not recorded”, and a consuming application cannot tell them apart. At this scale that is manageable; across a federation of independently-maintained catalogues it is exactly the interoperability obstacle Bue et al. (2023) describe.

5.2.1 Data integrity and processing time

Two further measures were added to characterise how the pipeline would behave at full catalogue scale, where records are ingested in bulk rather than individually reviewed.

The first checks for duplicate catalogue identifiers. This is a correctness check rather than a statistic: `work_id` is the database primary key, so a duplicated identifier would silently overwrite an existing record rather than raise an error. Across the corpus, 24 works produced 24 distinct identifiers with no collisions. At hand-curated scale a clean result is expected rather than surprising, and it is reported here because the failure mode it guards against is silent — precisely the kind that would go unnoticed during a bulk ingest.

The second measures processing time. On the deployment server, the full 25-file corpus parses in 0.006 seconds, a mean of 0.3 milliseconds per work. Because parsing is linear in file count and performs no cross-file work, this extrapolates to roughly 0.2 seconds for the full ~750-work Nielsen catalogue. The projection assumes the remainder of the catalogue resembles this sample in record size and complexity, which has not been verified and is the main caveat on the figure. Subject to that assumption, transformation speed is not a constraint on scaling: the limiting factor is obtaining the source data, not processing it.

5.3 The discovery engine: a negative result and its correction

The discovery engine was tested by asking whether it ranks known-related works above unrelated controls. Two Nielsen songs — settings of the same poet, in the same key — were tested against a control pairing one of those songs with a symphony in a different key.

Version 1 failed. It scored the unrelated pair 0.300 and the related pair 0.200, ranking them in precisely the wrong order. The engine was not weakly discriminating; it was actively misleading.

This result was investigated rather than set aside, and the investigation is what produced the fix. Inspecting the underlying data revealed that the two songs share a uniform quaver rhythm while the symphony incipit mixes crotchets and quavers — a strong distinguishing signal that pitch-only

comparison discarded. Version 2 incorporates rhythm and local bigram overlap as described in §4.2, and reverses the result.

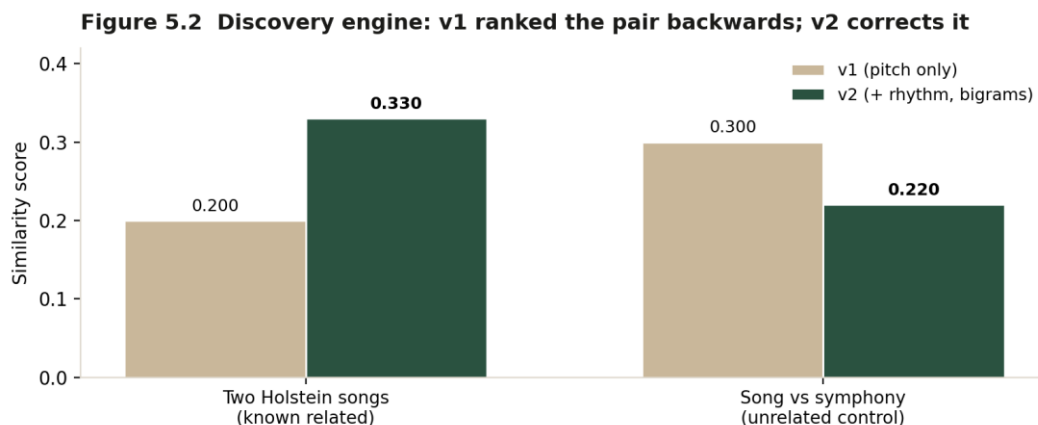


Figure 5.2 Version 1 ranked the related pair below the control; version 2 corrects the ordering.

The distinction between fixing a method and fitting a result matters here. The improvement comes from incorporating a real musical property the first version ignored, not from adjusting weights until the target case passed. The difference is testable, which is why a second validation case was run.

5.3.1 A second case that does not pass

The second test used two Delius works that the sources describe as companion pieces sharing folk-song material and almost always performed together — an independently documented relationship, chosen before the result was known. Against an unrelated control, the companion pieces scored 0.300 and the control also scored 0.300: an exact tie.

This is reported as it stands. The weights were not adjusted to separate the pair, because doing so would have fitted one example rather than improved the method, and would have invalidated the first result as evidence. The honest conclusion is bounded: version 2 corrects a specific, diagnosed failure mode and generalises to the rhythmic pattern it addresses, but it does not yet reliably separate every documented relationship at five-to-six-note incipit lengths.

The tie is itself informative. It supports the finding in §5.2 that MEI incipits, encoded to let a human recognise a work at a glance, carry too little information for dependable automated comparison. That is a limitation of the source encoding for this purpose, not solely of the algorithm — and it is a finding

this project is positioned to make precisely because it attempted the computation rather than assuming it would work.

5.4 Automated testing

Testing was treated as a deliverable, since Bue et al. (2023) identify the absent test suite as a principal cause of MerMEId's maintenance difficulty. The suite comprises 78 tests across four levels, all passing.

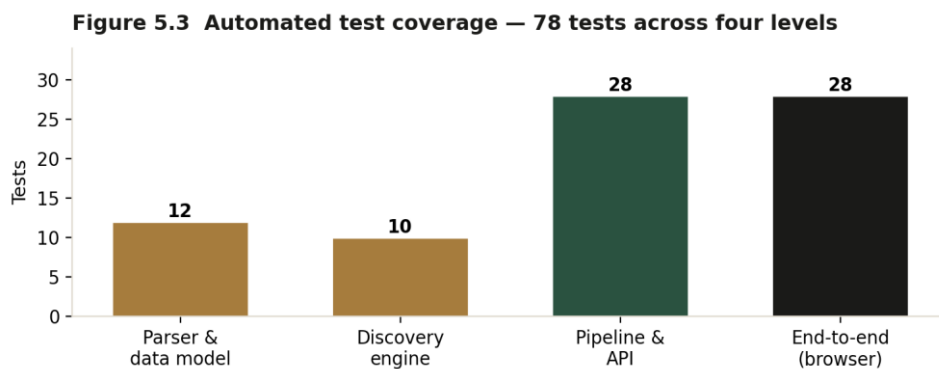


Figure 5.3 Test distribution across four levels; the browser tests exercise the real interface.

The end-to-end tests drive a real browser against a running server, verifying that notation renders with the expected number of note heads, that filters combine and persist across sorting, that navigation moves focus correctly, and that work entries are keyboard-operable. These are behaviours unit tests cannot reach.

The suite demonstrated its value by catching defects rather than merely confirming success. Three are worth recording: the coverage bars on the findings page rendered at zero width because span elements ignore width unless displayed as blocks; undated works sorted to the top of chronological listings; and linked-data identifiers broke catalogue detection. Each is now covered by a regression test. Static analysis — ruff and mypy under a configuration that requires type annotations — passes cleanly, having surfaced 23 genuine typing gaps that were fixed rather than suppressed.

5.5 Assessment against the project's aims

Aim	Outcome	Evidence
Transform MEI XML into a queryable store	Achieved	24 works, 96.0% robustness, 100% core-field completeness
Provide a documented REST API	Achieved	11 endpoints, OpenAPI schema generated from code, 50 backend tests
Accessible interface without XML knowledge	Achieved	Faceted browse, notation, audio; WCAG-aligned, keyboard operable
Working discovery feature	Partially achieved	Corrects a diagnosed failure; second validation case ties (§5.3.1)
Critical evaluation of the MEI model	Achieved	Evidence-based findings and four recommendations; live findings page
Explore multiple catalogues together	Achieved	CNW and DCW ingested by identical code; cross-composer similarity results

Table 5.2 Each aim assessed against evidence, including where the outcome was partial.

5.6 Limitations

Four limitations are material and are stated plainly.

- Corpus scale. The evaluation covers 24 works against roughly 750 in the Nielsen catalogue alone. The MerMEId export interface being unavailable, records were reconstructed in MEI 4.0 from published catalogue data — accurate, but a sample rather than a census. The completeness percentages are therefore indicative and would require confirmation at full scale. The timing measurement in §5.2.1 suggests the constraint is obtaining the data rather than processing it, but that projection is itself untested against the full catalogue.
- Discovery generalisation. As §5.3.1 records, the engine's correction is demonstrated on one diagnosed failure mode and does not extend to a second documented pair. It should be read as a working method with bounded evidence, not a validated one.

- No user study. The interface is validated by automated behavioural testing and against accessibility standards, but no study with representative users was conducted. Task completion, comprehension and perceived usefulness therefore remain untested, and Chapter 2 notes that no such study exists in the literature either.
- Single-instance deployment. The system runs on one virtual machine with no redundancy, monitoring or automated backup. For a project whose argument concerns longevity (§6.3) this is a fair criticism: the deployment demonstrates that the software is deployable, not that this particular instance is durable. The configuration is version-controlled and reproducible, so recreating it elsewhere is a scripted operation rather than a rediscovery, which is the property that actually matters for the argument being made.

5.6.1 The user study: design and ethical position

Because the absence of a user study is the most consequential limitation above, the study that was designed but not run is set out here, both to show that the gap was scoped rather than overlooked and because the design is itself a small contribution to a literature that contains no such study.

The intended design was a task-based usability evaluation with six to eight participants drawn across the four user groups in Table 3.1: at least two with a music background to represent scholars and students, at least two developers to represent API consumers, and the remainder general users with no domain knowledge. Participants would attempt four predefined tasks, each with an unambiguous success condition — locating all works for a given instrument, finding a specific work by catalogue number, switching catalogue and finding an orchestral work, and following a similarity suggestion from one work to another. Task completion and time would provide the quantitative measure; a System Usability Scale questionnaire would give a comparable overall score; and think-aloud commentary would supply the qualitative reasoning that completion rates alone cannot capture.

Two design decisions are worth stating. The tasks were specified before any results existed, so that the study could produce a negative finding rather than confirm the interface. And SUS was chosen

over a bespoke questionnaire precisely because it is standardised: a project-specific instrument would produce a number that could not be compared with anything, which for a system whose usability claims are otherwise untested would be of little value.

The study would also have addressed a gap that automated testing structurally cannot. Accessibility here is verified against WCAG through automated tests, and that verification did find two genuine failures — focus stranded on a hidden view during navigation, and two simultaneous main landmarks — both invisible to a sighted mouse user. But conformance is a proxy for usability, not evidence of it, and no disabled participant ever used the system. Only recruitment including disabled users would test whether the interface is usable rather than merely conformant.

Ethically, a study of this kind is low-risk but not risk-free, and it would require the standard safeguards. Participation would be voluntary and informed, with participants told in advance what the session involved, how long it would take, and that they could withdraw at any point without giving a reason. No personal data beyond broad user-group category would be recorded, responses would be stored without identifying information, and no session would be recorded without explicit consent. Participants would be told plainly that the system, not they, was under evaluation — a framing that matters in task-based studies, where failure to complete a task can otherwise be experienced as personal failure and suppress the honest reporting the study depends on. Institutional ethical approval would be sought before any participant was approached, and the study would not proceed without it.

5.7 What the evaluation shows

The evaluation supports a specific and bounded conclusion. The project's central technical risk — reliably parsing a complex, inconsistently-populated data model — is retired, with evidence. The access problem the project set out to solve is addressed: catalogue data that was unreachable is now browsable, searchable, machine-readable and exportable, across two catalogues through a single interface. The original contribution, melodic discovery, is genuinely partial: it works where the

diagnosed failure mode applies and does not yet generalise, and the reason is now understood well enough to state as a finding about the source encoding rather than merely a defect in the code.

6. Conclusion

This project set out to restore access to thematic catalogue data that had become unreachable when the software publishing it stopped working. That aim has been met, and met publicly: the system is deployed and reachable on the open web at the address given on the title page. Twenty-four works across two composer catalogues are transformed from MEI XML into a queryable store, served through a documented REST API, and presented in an accessible interface that requires no technical knowledge — with manuscripts, performance history, musical notation, audio playback, narrative composition histories, and outward links to Wikidata, MusicBrainz and IMSLP.

6.1 Contributions

Three contributions are worth distinguishing. The first is practical: a working, tested, documented system that makes stranded scholarly data usable again, built so that a third catalogue requires no code change. The second is evidential: the project answers empirically a question the literature leaves open, showing with data that MEI's optional-element flexibility produces uneven coverage that consuming applications cannot interpret, and that catalogue incipits are too short for dependable automated comparison. The third is methodological, and follows from having attempted the computation rather than assuming its outcome.

6.2 On the negative result

The discovery engine's history is the part of this project most worth reflecting on. A first version failed its own validation and ranked works in the wrong order. Investigating rather than discarding that failure identified the missing signal, and the corrected version reverses the result. A second validation case then tied, and was reported unchanged.

Retaining that tie was a deliberate decision. Adjusting the weights until it passed would have been straightforward and would have produced a more flattering chapter, but it would have converted evidence into decoration — the second case would no longer have tested anything, and the first result

would have lost its meaning as independent confirmation. The bounded claim that survives is smaller than the claim that was hoped for, and it is one a reader can actually rely on.

6.3 A broader theme: building for disappearance

The finding that most exceeds this project's immediate scope is Mathiak et al.'s: 376 of 466 digital scholarly editions gone, an average lifetime of 8.5 years. MerMEId was not carelessly built. It was built by competent people at a major institution, using technology defensible at the time, and it still became unreachable within a decade.

The implication is uncomfortable for how such projects are usually justified. Sustainability is not primarily a funding problem to be solved with continuation grants; it is an architectural property decided at design time. Every choice made here — a mainstream stack over a specialised one, no build step in the interface, the original XML retained as the authoritative source, containerised deployment, tests as a deliverable rather than an afterthought — is a bet on institutional support eventually ending. That assumption should probably be the default in digital humanities software rather than a distinguishing feature of it.

Those choices are now demonstrated rather than merely asserted. The system runs on a public server, deployed by a version-controlled script, using only tools that ship with the operating system. The claim is modest but real: not that this instance will still be running in ten years — no single student VM should be expected to — but that the knowledge required to stand it up again is written down and executable rather than resident in one person's memory. That is the property MerMEId lost, and the one worth designing for.

There is a related point about data and its routes. The MEI files survived perfectly; only the path to them broke. Preservation policy in this field concentrates heavily on formats and repositories, and comparatively little on access layers, which are treated as replaceable infrastructure. This project is a small demonstration that the access layer is exactly where the failure occurred, and that rebuilding it is tractable when the underlying data is sound and open.

6.4 Further work

Four directions follow directly from the limitations in §5.6. Ingesting the full Nielsen catalogue would convert indicative completeness figures into definitive ones and would test the pipeline at a scale that may expose encoding variation this corpus does not contain. Extending the discovery engine — longer excerpts where available, n-gram indexing, and validation against a larger set of documented relationships — would establish whether the method generalises or whether incipit length is a genuine ceiling. A user study with participants from the four groups in Table 3.1 would test the comprehension and usefulness claims that automated testing cannot reach, and would contribute the first such study to a literature that lacks one. Finally, the linked-data identifiers already embedded in every record make a future RDF migration a contained piece of work rather than a rewrite, which would connect these catalogues to the wider FAIR ecosystem Weigl et al. (2019) describe.

6.5 Closing

What began as a software problem turned out to be partly an evidential one. Rebuilding the bridge to this data required parsing the MEI model closely enough to discover where it is inconsistent, and attempting melodic discovery closely enough to discover where the encoding runs out. Those findings were not the objective, but they may prove as durable as the system itself — and both were only available because the work was carried through to the point where it could fail informatively.

References

- Bue, M., Gammert, J., Gubsch, C., Jettka, D., Kolb, S., Lewis, D., Neumann, J., Richter, M., Richts-Matthaei, K., & Stadler, P. (2023). Transitional MerMEId – Responding to community needs in software and sustainability. In *TEI/MEC Conference*. Paderborn. <https://teimec2023.uni-paderborn.de/contributions/143.html>
- Geertinger, A. T., & Lundberg, S. (2013). MerMEId: Creating thematic catalogues using MEI metadata. In *Music Encoding Conference 2013*. Mainz. <https://pure.kb.dk/en/publications/mermeid-creating-thematic-catalogues-using-mei-metadata>
- IMSLP. (n.d.). *International Music Score Library Project (Petrucci Music Library)*. Retrieved September 19, 2026, from <https://imslp.org>
- Krabbe, N., & Geertinger, A. T. (2012). MEI (Music Encoding Initiative) as a basis for thematic catalogues. In *RISM Conference on Music Documentation in Libraries, Scholarship, and Practice*. https://rism.info/resources-old-website/community-content/events/RISM_Conference_2012/TeichGeertinger_Final.pdf
- Lubanovic, B. (2023). *FastAPI: Modern Python web development*. O'Reilly Media.
- Mathiak, B., Neuefeind, C., Schildkamp, P., Karadkar, U., Stigler, J., Steiner, E., Vasold, G., Tosques, F., Ciula, A., Maher, B., Newton, G., Arneil, S., & Holmes, M. (2020). Sustainability strategies for digital humanities systems. In *Digital Humanities Conference (DH2020)*. Alliance of Digital Humanities Organizations. https://dh2020.adho.org/wp-content/uploads/2020/07/565_SustainabilityStrategiesforDigitalHumanitiesSystems.html
- MetaBrainz Foundation. (n.d.). *MusicBrainz: Open music encyclopedia*. Retrieved September 19, 2026, from <https://musicbrainz.org>

Ramírez, S. (n.d.). *FastAPI* [Computer software]. Retrieved September 19, 2026, from <https://fastapi.tiangolo.com>

Weigl, D. M., Goebel, W., Crawford, T., Gkiokas, A., Gutierrez, N. F., Porter, A., Santos, P., Karreman, C., Vroomen, I., Liem, C. C. S., Sarasúa, A., & van Tilburg, M. (2019). Interweaving and enriching digital music collections for scholarship, performance, and enjoyment. In *Proceedings of the 6th International Conference on Digital Libraries for Musicology (DLfM)* (pp. 84–88). The Hague.

Declaration on the Use of AI

This report was written by the author. An AI assistant (Claude, Anthropic) was used to help polish wording, improve clarity, and refine structure and formatting, and to assist in scaffolding and reviewing project code. All design decisions, technical choices, research interpretation, evaluation methods and conclusions are the author's own. The cited sources and all reported results were reviewed and verified by the author.